

0.1 Introduction and Problem Statement

Grasp preshaping is the problem of determining the optimal grasp type, hand pose, wrist orientation, and finger closure amount **before** the hand contacts the object. In the context of the Mia hand prosthesis, this must happen in real time as the user approaches an object, using only the partial point cloud observed from one or more depth cameras.

The core challenges are:

- Only the front face of objects is observed (partial point cloud) — the backside is unknown
- A real-time budget of approximately 50 ms from point cloud to grasp command
- A high-dimensional search space: 6D hand pose + grasp type + wrist rotation + closure amount
- Diverse object geometries must be handled with a single model, without per-object training

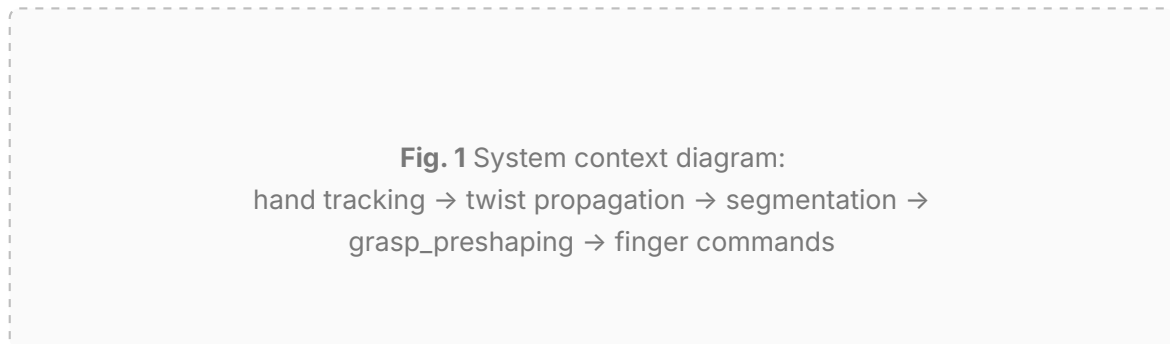


Figure 1: System context diagram showing the grasp preshaping pipeline's position in the overall system.

0.1.1 Related Work and Positioning

Several established approaches to grasp planning exist, but none directly apply to our setting:

- **Dex-Net** (Mahler et al., 2017): sampling-based grasp planning on full 3D object models. Requires a complete object mesh and GQ-CNN inference — not feasible when only a partial point cloud is available in real time.
- **Contact-GraspNet** (Sundermeyer et al., 2021): deep learning-based grasp prediction from point clouds. Requires GPU inference (100 ms+), large-scale training data, and outputs grasps for generic parallel-jaw or suction grippers — not the anthropomorphic Mia hand.
- **GraspNet-1Billion** (Fang et al., 2020): model-based grasp sampling on full scene point clouds. Again requires complete point clouds and is designed for parallel-jaw grippers.

These approaches share one or more limitations that make them unsuitable for our system: they require complete point clouds, need GPU-accelerated inference, assume parallel-jaw grippers, or depend on large training datasets. Our system must operate on partial point clouds, run on embedded hardware within 50 ms, and plan grasps for the multi-fingered Mia hand with coupled kinematics — for which no training data exists.

This leads us to a **geometry-first approach** that combines a truncated signed distance field (TSDF) for efficient collision queries with a pre-computed finger kinematics lookup table, optimized via sequential Monte Carlo sampling. The key insight is to plan in **task space** using the TSDF (not in joint space or object space), which decouples the planner from specific hand kinematics at runtime.

0.2 Pipeline Architecture Overview

The pipeline processes a segmented point cloud through the following stages:

```
PointCloud → Prune(ROI) → Morton Sort → TSDF Construction →  
Superquadric Backside Estimation → SMC Optimization Loop →  
Score Particles → Select Best Grasp → Output
```

Fig. 2 Pipeline block diagram
with approximate timing per stage.

Highlight: **TSDF is built once and reused** across all SMC iterations.

Figure 2: Pipeline block diagram with timing annotations per stage.

The core library is implemented in Rust for zero-cost abstractions, no garbage collection pauses, memory safety, and seamless C FFI. A thin C++ ROS 2 bridge node handles message passing. The modules are:

Module	Responsibility	File
<code>c_api</code>	FFI entry point, orchestrates full pipeline	<code>c_api.rs</code>
<code>predictor</code>	Motion model sampling, SMC particles, twist $\rightarrow$ SE(3)	<code>predictor.rs</code>
<code>pointcloud_helper</code>	Pruning, Morton ordering, TSDF construction	<code>pointcloud_helper.rs</code>
<code>planner</code>	Grasp scoring (3 types), sweep-for-collision, binary refinement	<code>planner.rs</code>
<code>superquadric</code>	Shape completion for unobserved backside	<code>superquadric.rs</code>
<code>lut_helper</code>	Finger contact LUT, dual quaternion math	<code>lut_helper.rs</code>
<code>runtime_config</code>	YAML-based runtime config with compile-time defaults	<code>runtime_config.rs</code>
<code>debug_export</code>	NPZ debug dump writer	<code>debug_export.rs</code>
Bridge node	ROS 2 service, subscriptions, FFI calls	<code>preshaping_service_bridge_node.cpp</code>

Table 1: Module responsibilities and file locations.

0.3 Region of Interest and TSDF Construction

0.3.1 ROI Prediction

- Input: current hand pose + twist (6D velocity) + covariance
- Sample future hand poses over a 5 s prediction horizon using a screw motion model
- Twist $\rightarrow$ SE(3) via the exponential map (`twist_to_se3`)
- Add Gaussian noise scaled by `sqrt(t)` and covariance
- AABB of sampled fingertip positions $\rightarrow$ ROI, inflated by hand radius (0.05 m), clamped to [0.1 m, 0.5 m]

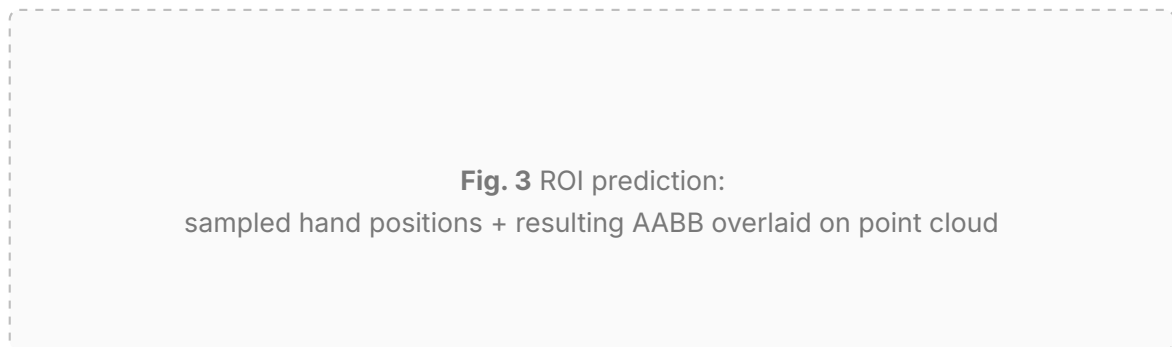


Figure 3: ROI prediction visualization with sampled hand positions and resulting bounding box.

0.3.2 Point Cloud Pruning and Morton Ordering

Only points within the ROI are retained, reducing data before the expensive TSDF construction. The pruned points are sorted by Morton code (Z-order curve) to improve spatial locality for the subsequent BFS.

0.3.3 TSDF Construction via BFS

An unsigned truncated distance field is built using BFS from surface voxels — each voxel gets its exact distance to the nearest surface. The truncation band is `TRUNCATION_CELLS = 4` (20 mm at 5 mm resolution), keeping the representation compact.

Sign determination: for each voxel, all cameras are checked. If the voxel is behind the surface and the voxel-to-surface direction aligns with a camera ray, the voxel is marked negative (inside the object).

Fig. 4 2D TSDF cross-section:
truncation band, positive/negative regions,
zero-crossing, camera rays

Figure 4: 2D TSDF cross-section showing truncation band, positive/negative regions, zero-crossing, and camera rays.

0.3.4 Design Decisions

- **Why TSDF (not mesh/raw points):** $O(1)$ distance queries + smooth gradients → efficient collision detection and surface normals. Raw point clouds require nearest-neighbour searches per contact point, which is too slow for 20 000 particles.
- **Why BFS (not projective TSDF):** BFS gives analytically exact nearest-surface distances. Projective TSDF was initially considered but produces staircase artifacts at oblique surfaces and its accuracy depends on camera resolution and viewing angle. BFS works correctly with any number of cameras.
- **Why 5 mm resolution:** distinguishes finger-sized features while keeping cost manageable ($60^3 = 216K$ voxels for a 0.3 m ROI). Coarser resolutions (10 mm) fail to detect thin objects; finer resolutions (2 mm) increase voxel count by 15×.

0.4 Superquadric Backside Estimation

0.4.1 The Problem

Cameras only see the front face of objects. In the TSDF, unobserved regions are set to `f32::MAX` — “shadow regions” where the planner cannot detect collisions, allowing grasps to pass straight through the object.

0.4.2 The Solution

A superquadric is fitted to the observed point cloud to estimate distances in unobserved regions, filling the shadow regions with geometrically plausible distance values.

0.4.3 Pipeline Steps

1. **PCA orientation**: principal components of the point cloud define the SQ axes. Rotation is locked (not optimized) — PCA is reliable for most objects and avoids the convergence risk of optimizing all 11 SQ parameters.
2. **OBB initial guess**: project points onto PCA axes → initial scale (a, b, c) and translation.
3. **Parallel template matching** (rayon): three templates are fitted simultaneously, and the best fit is selected:
 - Sphere: $\varepsilon_1 = 1.0$, $\varepsilon_2 = 1.0$
 - Box: $\varepsilon_1 = 0.1$, $\varepsilon_2 = 0.1$
 - Cylinder: $\varepsilon_1 = 0.1$, $\varepsilon_2 = 1.0$
4. **Gauss-Newton refinement**: 4 iterations with Levenberg-Marquardt damping on 6 parameters (3 scale + 3 translation), rotation locked from PCA.
5. **TSDF fusion** — three domains (see Figure 6):
 - **Domain A** (Camera Authority): near the observed surface — camera data wins.
 - **Domain B** (SQ Authority): unobserved regions (`f32::MAX`) — Taubin distance fills the gaps.
 - **Domain C** (Blending Zone): at the boundary — smoothstep blending ensures continuity.

0.4.4 Design Decisions

- **Why superquadrics (not deep learning)**: the 50 ms budget excludes neural network inference; SQ fitting completes in 1–2 ms.
- **Why only 3 templates**: early testing with 5 templates showed less than 2% improvement in fit error for a 60% increase in fitting time. The three chosen templates (sphere, box, cylinder) cover the vast majority of graspable household objects.
- **Why one-directional sign override**: the SQ can only **add** interior (positive → negative), never remove it. This prevents holes near the observed surface where the SQ and camera surfaces are slightly misaligned.
- **Why `SQ_MIN_SIGN_OVERRIDE_CELLS = 2`**: voxels within 2 cells of the surface retain the camera sign, preventing artifacts from slight SQ-camera misalignment at the boundary.
- **Fallback**: if too few points are available or the fit error is high, backside estimation is skipped entirely and the pipeline proceeds with camera-only TSDF (graceful degradation).

Fig. 5 Three superquadric templates with epsilon values and example fitted shapes

Figure 5: Three superquadric templates (sphere, box, cylinder) with epsilon values and example shapes.

Fig. 6 TSDF cross-section with Domain A/B/C color-coded, showing backside fill

Figure 6: TSDF cross-section showing Domain A (camera authority), B (SQ authority), and C (blending zone) color-coded.

Parameter	Default Value
SQ_MAX_GN_ITERATIONS	4
SQ_GN_DAMPING	0.1
SQ_BLEND_DELTA_CELLS	3
SQ_MIN_FIT_POINTS	20
SQ_FIT_ERROR_THRESHOLD	0.15
SQ_MIN_SIGN_OVERRIDE_CELLS	2
SQ_ENABLE_BACKSIDE	true

Table 2: Superquadric configuration parameters with default values.

0.5 Finger Contact Lookup Table (LUT)

A pre-computed table maps (contact point, closure amount) $\rightarrow$ SE(3) transform, capturing the full Mia hand kinematics offline.

- **30 anatomical contact points:** index joints (8), middle/ring/little fingertips (10), thumb adduction (3), thumb abduction (3), palm (4)
- **Closure:** a single scalar [0.0, 1.0] parameterizing coupled finger motion
- **Storage:** dual quaternions at discrete samples, with linear interpolation between them
- **Thumb modes:** adduction (lateral/key grip) vs. abduction (opposition, cylindrical/pinch) — separate tables
- **Max closure per grasp type:** pre-computed self-collision limits stored in the LUT; the sweep is clamped to these limits
- **Why LUT (not runtime FK):** O(1) lookup, no kinematics library dependency at runtime, and dual quaternions naturally handle the coupled motion of the Mia hand's linked finger joints

Fig. 7 Hand model with labeled contact points; three grasp types with active contacts highlighted

Figure 7: Hand model with labeled contact points and three grasp types with active contacts highlighted.

0.6 Grasp Scoring and Collision Detection

0.6.1 Sweep-for-Collision and Binary Refinement

Given a hand pose and grasp type, finger contacts are swept from open (closure = 0) to closed (closure = max). The first TSDF collision (distance < `collision_tol` = 10 mm) determines the closure amount. A subsequent binary search refines the closure to within `BINARY_SEARCH_TOL` (5%). Locked points (palm) are checked at sample 0 — if already colliding, the grasp is assigned Tier 3.

0.6.2 The Four-Tier Scoring System

Tier	Con- di- tion	Score	Mean- ing
Tier 1	Col- li- sion + $\geq$ min_contacts + $\geq$ min_fingers (thumb + in- dex)	$0.8 + 0.2 \times \text{contact_fraction}$	Valid grasp
Tier 2	Col- li- sion but too few fin- ger groups or miss- ing thumb/ in- dex	$0.25 \times \text{contact_fraction} + 0.25 \times \text{proximity_penalty}$	Soft re- jec- tion — gives gra- di- ent
Tier 3	Col- li- sion at clo- sure = 0 (palm already in- side ob- ject)	0.1	Bad start- ing po- si- tion
Tier 4	No col- li- sion	0.0-0.05 (proximity score)	Hand misses ob- ject

Table 3: The four-tier scoring system with conditions, score formulas, and interpretations.

The tiered system is critical because a binary hit/miss score provides **zero gradient** for optimization. The four tiers create a smooth landscape that guides the SMC optimizer from “approaching the object” through “making contact” to “forming a stable grasp” (see Figure 8).

- **Tier 4 proximity**: a sparse check (palm + index tip + thumb tip at mid-closure) — approximately 8× cheaper than a full sweep, yet provides directional gradient toward the object.
- **Tier 2 proximity penalty**: measures how close the missing fingers are to the surface, giving the optimizer gradient to improve finger placement.

0.6.3 Active Contact Analysis

- `find_active_contacts`: filters contacts to those near the zero-crossing ($|\text{dist}| < 2 \times \text{collision_tol}$), rejecting deep-interior contacts from negative TSDF regions. Without this filter, grasps that penetrate deeply into the object without touching the surface would receive incorrectly high scores (see Table 6, challenge 2).
- `compute_alignment`: dot product of finger closing direction vs. surface normal — measures whether the fingers are “pushing into” the surface.
- `compute_force_closure`: $1 - |\text{mean normal}|$ — well-distributed opposing contacts score higher, favouring stable grasps.

0.6.4 Combined Score

```
combined = (w_prob × probability + w_align × alignment + w_fc ×
force_closure + w_contact × contact_score)
          / (w_prob + w_align + w_fc + w_contact)
```

Default weights: probability = 1.0, alignment = 1.0, force_closure = 1.0, **contact_score = 3.0**. Contact score is weighted 3× because it encodes the most important signal: whether the hand is actually forming a grasp.

Fig. 8 Four-tier scoring landscape
vs. closure amount — annotate with example hand configs at each tier

Figure 8: Four-tier scoring landscape plotted against closure amount, with example hand configurations at each tier.

Grasp Type	Score Con- tacts	Min Contacts	Min Fingers	Thumb Mode
Cylindrical	17	3	thumb + index	Abduction
Pinch	2	2	thumb + index	Abduction
Lateral	5	2	thumb + index	Adduction

Table 4: Grasp type specifications: score contacts, minimum contacts, minimum fingers, and thumb mode.

0.7 Sequential Monte Carlo (SMC) Optimization

0.7.1 Why Optimization

The search space has approximately 9 effective degrees of freedom (6D pose + grasp type + wrist rotation + closure). Exhaustive search is infeasible, and the motion model provides only a prior — the optimal grasp requires iterative refinement.

Alternatives were considered and rejected:

- **Cross-Entropy Method (CEM):** prototyped early on; produced similar-quality results with more implementation complexity. SMC’s particle resampling maps more naturally to our parallel scoring architecture.
- **Bayesian optimization:** overkill for this budget, does not parallelize well across 20 000 particles.
- **Grid refinement:** cannot handle rotation parameters gracefully.
- **Brute-force with more samples:** wastes compute evaluating poor regions of the search space.

0.7.2 SMC Algorithm

1. **Iteration 0:** 20 000 particles sampled from the motion model (twist + covariance); random grasp type; random wrist rotation $\in [-\pi/2, \pi/2]$.
2. **Score all particles:** parallel via rayon.
3. **Select elites:** top 5% (1 000) by combined score.
4. **Resample:** 20 000 new particles around elites with Gaussian jitter; standard deviation decays as `initial_std × 0.7iteration`.
5. **Repeat:** up to 5 iterations; early stop if best score changes less than 0.01 after a minimum of 3 iterations.

0.7.3 Key SMC Enhancements

Each enhancement was added to solve a specific observed problem during development:

- **Elite injection** (top 1% preserved as exact copies):

- Problem: the best grasp found in an early iteration was lost during resampling — the ROS node reported a worse grasp than the debug visualizer.
 - Solution: carry the best-so-far forward unchanged across iterations.
- **Score-weighted parent selection:**
 - Problem: uniform elite selection does not exploit the best-known regions.
 - Solution: parents are chosen proportional to their score, bounded to $[0, 1]$ so no single elite dominates.
- **Grasp type inheritance with 10% mutation:**
 - Problem: “type collapse” — one grasp type dominates early and eliminates exploration of alternatives.
 - Solution: 90% inherit the parent type (exploitation), 10% mutate to a performance-weighted random type (exploration).
- **Wrist rotation perturbation** (not re-randomization):
 - Problem: re-randomizing the wrist angle each iteration destroys convergence.
 - Solution: perturb the parent’s wrist angle with a decaying Gaussian ($\pm 17^\circ$ early $\rightarrow \pm 1.4^\circ$ late).
- **sample_probability = 1.0 for resampled particles:**
 - Problem: the inherited motion model probability is meaningless for jittered particles whose position no longer comes from the twist prediction.
 - Solution: set to 1.0, so the score reflects purely grasp quality after iteration 0.

0.7.4 Computational Budget

The TSDF is built once (dominant cost); each iteration only re-samples poses and re-scores. 20 000 particles are scored in parallel per iteration. The target is less than 50 ms total.

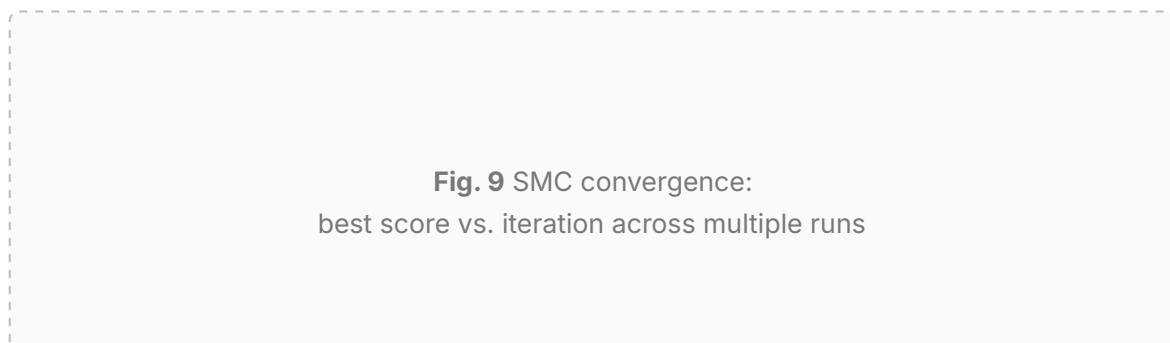


Figure 9: SMC convergence plot showing best score vs. iteration across multiple runs.

Fig. 10 Particle distribution at iterations 0, 2, 4 — spatial clustering as variance decays

Figure 10: Particle distribution at iterations 0, 2, and 4 showing spatial clustering as variance decays.

Parameter	Value
ITERATIONS	5
DECAY_RATE	0.7
ELITE_RATIO	0.05
ELITE_PRESERVE_RATIO	0.01
GRASP_TYPE_MUTATION_RATE	0.1
SMC_CONVERGENCE_TOL	0.01
SMC_MIN_ITERATIONS	3
PREDICTION_SAMPLES	20000
INITIAL_PROPOSAL_STD_V	0.002
INITIAL_PROPOSAL_STD_OMEGA	0.005
INITIAL_PROPOSAL_STD_WRIST	0.3

Table 5: SMC configuration parameters with default values.

0.8 Results and Evaluation

Note: This section contains placeholder structure. Fill in with data from debug dumps and experimental runs.

0.8.1 Timing Breakdown

[TODO: Insert timing data from debug dumps or instrumentation.]

Expected structure: a table or bar chart showing milliseconds spent in each pipeline stage:

- ROI pruning
- Morton sort
- TSDF construction (BFS + sign determination)
- Superquadric fitting
- SMC iteration 0 scoring
- SMC resampled iterations (1–4)
- Total

Target: total under 50 ms. If profiling data exists, include a figure:

Fig. 12 Timing breakdown:
stacked bar or table of ms per pipeline stage

Figure 11: Timing breakdown of the grasp preshaping pipeline, showing milliseconds spent in each stage.

0.8.2 Grasp Success Rate

[TODO: Insert grasp trial data — simulated or real.]

If available, include a table of objects tested, grasp type selected, and success/failure. Even anecdotal results (e.g., “tested on 15 household objects, successful grasp on 12”) are valuable.

0.8.3 Ablation: Superquadric Backside Estimation

[TODO: Show 1–2 examples where SQ backside estimation prevents a grasp from passing through the object.]

This directly justifies Figure 6 and the three-domain fusion approach. Without backside estimation, the TSDF has `f32::MAX` in shadow regions, and the planner cannot detect collisions on the far side of the object.

Fig. 13 Ablation comparison:
grasp with and without SQ backside estimation.
Without SQ, fingers pass through the object.

Figure 12: Ablation comparison showing grasp planning with and without superquadric backside estimation.

0.8.4 SMC Convergence Analysis

[TODO: Describe the typical convergence curve from Figure 9.]

Expected content:

- How many iterations until convergence (typically 3–4 out of 5)?
- How much does the best score improve from iteration 0 to final (e.g., 0.3 → 0.85)?
- Does elite injection consistently preserve the best grasp?

0.9 System Integration

The Rust core library is called from a C++ ROS 2 bridge node (`preshaping_service_bridge_node.cpp`) via a C ABI. The bridge subscribes to `/hand_pose`, `/hand_twist`, and the segmented point cloud, calls the Rust library on the `/grasp_preshaping/compute_grasp` service trigger, and publishes per-finger closure commands (thumb, index, MRL), wrist orientation, and the target hand pose. A `preshaping_closure_fraction` ROS parameter (default 0.3) scales the planned closure for preshaping (as opposed to full grasping). All tunable parameters are loaded from `config/grasp_preshaping.yaml` at runtime, with compile-time defaults as fallback. When debug visualization is enabled, the library exports an NPZ file containing the full TSDF, all scored grasps across all SMC iterations, fitted superquadric parameters, and the input pose/twist.

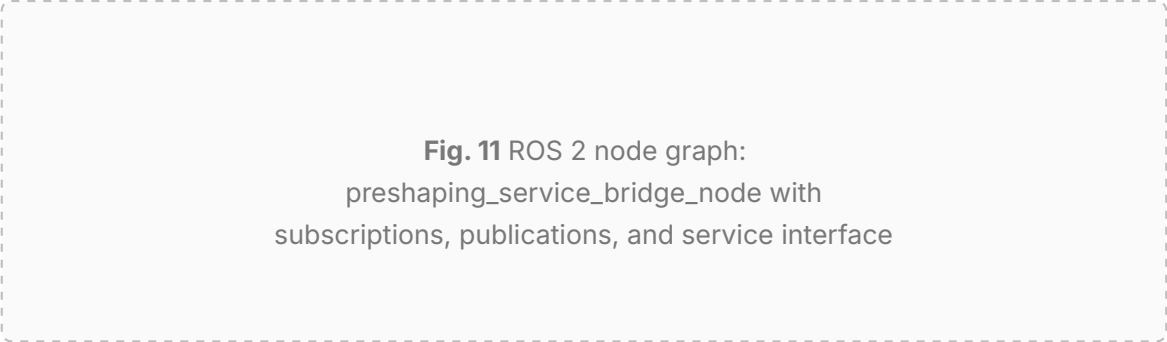


Fig. 11 ROS 2 node graph:
preshaping_service_bridge_node with
subscriptions, publications, and service interface

Figure 13: ROS 2 node graph showing the preshaping service bridge node with its subscriptions, publications, and service interface.

0.10 Lessons Learned

#	Challenge	Root Cause	Resolution
1	TSDF sign wrong with opposing cameras	Majority <code>behind_count > vote (n_cams/2)</code> fails with 2 opposite cameras: each sees the voxel as "in front" from its side, so <code>behind_count</code> only reaches 1	Changed to <code>inside_votes > 0</code> (any camera with good alignment confirms inside) + superquadric tiebreaker
2	High scores for grasps not touching surface	After sign fix, negative voxels (deep inside) triggered as "collisions" → contact score high despite no surface contact	Surface proximity filter in <code>find_active_contacts</code> : only count contacts with <code> dist < threshold</code> , reject deep-interior
3	SMC loses best grasp across iterations	ROS node selected from last iteration only; debug visualizer searched all iterations → different "best"	Track overall best across all iterations for output + elite injection to prevent loss
4	Closure fraction had no effect	Rust <code>PRESHAPING_CLOSURE_FRACTION</code> was dead code; actual control in C++ ROS param (default 0.3)	Removed dead Rust constant; centralized config via ROS parameter
5	SQ artifacts near observed surface	SQ surface slightly misaligned with camera surface → sign flips near boundary	One-directional override (only positive → negative) + <code>SQ_MIN_SIGN_OVERRIDE_CELLS = 2</code> preserves camera authority near surface

Table 6: Summary of engineering challenges encountered, their root causes, and resolutions.

Each challenge yielded a broader design insight:

- **Multi-camera TSDF sign determination** requires consensus logic that accounts for opposing viewpoints, not simple majority voting. The “any camera confirms inside” rule is more robust for arbitrary camera configurations.
- **Distance-based collision metrics alone are insufficient:** surface proximity filtering is essential to distinguish genuine contacts from interior penetration in signed distance fields.
- **Iterative optimization outputs** must be selected from the global best across all iterations, not just the final iteration — a lesson applicable to any multi-iteration sampling scheme.
- **Configuration parameters** must have a single source of truth; duplicate constants across language boundaries (Rust/C++) are a maintenance hazard.
- **When blending multiple geometric representations**, preserving the authority of higher-fidelity data near its source prevents boundary artifacts. The one-directional override rule is a general principle for sensor-model fusion.

0.11 Summary and Contributions

The grasp preshaping pipeline achieves real-time (50 ms) grasp planning from partial point clouds through a combination of:

- A BFS-constructed TSDF for $O(1)$ collision queries with exact distance values
- Superquadric shape completion for unobserved backsides, with graceful fallback to camera-only data
- A four-tier scoring system that creates a smooth optimization landscape across the full quality spectrum
- Sequential Monte Carlo optimization with five targeted enhancements for robust convergence
- A pre-computed finger contact LUT that eliminates runtime kinematics dependency
- Three grasp types (cylindrical, pinch, lateral) with automatic selection via SMC

The design prioritizes three properties:

- **Robustness:** graceful degradation at every stage (SQ fails → camera-only TSDF; early termination on convergence; fallback paths throughout)
- **Efficiency:** TSDF built once and reused, parallel scoring via rayon, Morton-ordered spatial data, LUT $O(1)$ lookups
- **Observability:** debug dumps capture the full state (TSDF, all scored grasps, SQ parameters) for offline analysis